SQL Query Challenges: Movies Database

A comprehensive collection of SQL queries ranging from basic to advanced levels, designed to test and improve your database querying skills. This presentation covers essential SQL operations including SELECT statements, JOINs, aggregations, window functions, and advanced analytical queries on a movie database schema.

Made with GAMMA

Basic Queries: Foundation Level

These foundational queries introduce basic SQL concepts including data retrieval, filtering, sorting, and simple aggregations. They serve as an excellent starting point for building SQL proficiency.

Q1: List All Movies

Retrieve all movies with their release year and IMDb rating, ordered by release year and title.

```
SELECT movie_id, title, release_year, imdb_rating
FROM movies
ORDER BY release_year, title;
```

Q2: Actors Born After 1980

Show all actors born after 1980, displaying actor ID, name, and birth year.

```
SELECT actor_id, name, birth_year
FROM actors
WHERE birth_year > 1980
ORDER BY birth_year;
```

Q3: Count Movies by Industry

Count how many movies belong to each industry (Bollywood/Hollywood).

```
SELECT industry, COUNT(*) AS movie_count
FROM movies
GROUP BY industry
ORDER BY movie_count DESC;
```

Q4: Retrieve All Languages

Display all languages available in the language table with language ID and name.

```
SELECT language_id, name AS language_name
FROM languages
ORDER BY name;
```

Q5: Movie Budget and Revenue

Display the budget and revenue of each movie with movie ID, title, budget, revenue, unit, and currency.

```
SELECT f.movie_id, m.title, f.budget, f.revenue, f.unit, f.currency
FROM financials f
LEFT JOIN movies m ON f.movie_id = m.movie_id
ORDER BY f.movie_id;
```

Made with GAMMA

Intermediate Queries: JOINs and Aggregations

These intermediate queries demonstrate the power of SQL JOINs, grouping operations, and calculated fields. They build upon the foundational concepts by combining data from multiple tables and performing more complex calculations.

Q6: Movies with Language Names

Show each movie along with its language name by joining the movies and languages tables.

```
SELECT m.movie_id, m.title, l.name
AS language_name
FROM movies m
LEFT JOIN languages l ON
m.language_id = l.language_id
ORDER BY m.movie_id;
```

Q7: Top 5 Highest-Rated Movies

Find the top 5 highest-rated movies based on IMDb rating, ordered by rating and release year.

```
SELECT movie_id, title, release_year,
imdb_rating
FROM movies
ORDER BY imdb_rating DESC,
release_year DESC
LIMIT 5;
```

Q8: Movies with All Actors

For each movie, list all actors who acted in it using GROUP_CONCAT to combine actor names.

```
SELECT m.movie_id, m.title,
GROUP_CONCAT(DISTINCT a.name
SEPARATOR ', ') AS actors
FROM movies m
LEFT JOIN movie_actor ma ON
m.movie_id = ma.movie_id
LEFT JOIN actors a ON ma.actor_id =
a.actor_id
GROUP BY m.movie_id, m.title
ORDER BY m.title;
```

Q9: Calculate Movie Profit

Calculate the profit (revenue − budget) for each movie, normalizing billions to millions.

```
SELECT f.movie_id, m.title, f.currency,
f.budget_m AS budget_in_millions,
f.revenue_m AS revenue_in_millions,
(f.revenue_m - f.budget_m) AS
profit_in_millions, CASE WHEN
(f.revenue_m - f.budget_m) > (SELECT
AVG(CASE WHEN LOWER(unit) LIKE
'%billion%' THEN (revenue * 1000 -
budget * 1000) ELSE (revenue -
budget) END) FROM financials) THEN
'YES' ELSE 'NO' END AS
above_avg_profit FROM (SELECT
movie_id, currency, unit, CASE WHEN
LOWER(unit) LIKE '%billion%' THEN
budget * 1000 ELSE budget END AS
budget_m, CASE WHEN LOWER(unit)
LIKE '%billion%' THEN revenue * 1000
ELSE revenue END AS revenue_m
FROM financials) f LEFT JOIN movies
m ON f.movie_id = m.movie_id
ORDER BY profit_in_millions DESC;
```

Q10: Post-2015 Movies with Revenue > Budget

Find all movies released after 2015 where revenue exceeds budget, using normalized values.

```
SELECT m.movie_id, m.title,
m.release_year, f.currency,
f.budget_million AS
budget_in_millions, f.revenue_million
AS revenue_in_millions
FROM movies m
JOIN financials f ON m.movie_id =
f.movie_id
WHERE m.release_year > 2015
AND f.revenue_million >
f.budget_million
ORDER BY (f.revenue_million -
f.budget_million) DESC;
```

Made with GAMMA

Advanced Queries: Part 1

These advanced queries demonstrate sophisticated SQL techniques including subqueries, window functions, CTEs (Common Table Expressions), and complex analytical operations that require multiple levels of data processing.

Q1: Most Prolific Actor

Find the actor who has acted in the highest number of movies.

```
SELECT a.actor_id, a.name,
COUNT(ma.movie_id) AS
movies_count
FROM actors a
JOIN movie_actor ma ON a.actor_id =
ma.actor_id
GROUP BY a.actor_id, a.name
ORDER BY movies_count DESC
LIMIT 1;
```

Q2: Revenue by Industry

Calculate the total revenue generated by Bollywood vs Hollywood movies, aggregating normalized revenue by industry.

```
SELECT m.industry, f.currency,
SUM(CASE WHEN LOWER(f.unit) LIKE
'%billion%' THEN f.revenue * 1000
ELSE f.revenue END) AS
total_revenue_in_millions
FROM movies m
JOIN financials f ON m.movie_id =
f.movie_id
GROUP BY m.industry, f.currency
ORDER BY total_revenue_in_millions
DESC;
```

Q3: Movies Above Industry Average

Identify movies where the IMDb rating is above the industry average.

```
SELECT m.movie_id, m.title,
m.industry, m.imdb_rating,
ind.avg_rating
FROM movies m
JOIN (SELECT industry,
AVG(imdb_rating) AS avg_rating
FROM movies GROUP BY industry)
ind ON m.industry = ind.industry
WHERE m.imdb_rating >
ind.avg_rating
ORDER BY m.industry, m.imdb_rating
DESC;
```

Q4: Highest ROI Movies

Find the movie with the highest ROI ((revenue − budget) / budget), computing ROI using normalized values in millions.

```
SELECT f.movie_id, m.title, f.currency,
budget_m, revenue_m,
ROUND((revenue_m - budget_m) /
budget_m, 4) AS roi
FROM (SELECT movie_id, (CASE
WHEN LOWER(unit) LIKE '%billion%'
THEN budget * 1000 ELSE budget
END) AS budget_m, (CASE WHEN
LOWER(unit) LIKE '%billion%' THEN
revenue * 1000 ELSE revenue END)
AS revenue_m, currency FROM
financials) f
JOIN movies m ON f.movie_id =
m.movie_id
WHERE f.budget_m > 0
ORDER BY roi DESC
LIMIT 10;
```

Q5: Movie Summary Report

Build a comprehensive movie summary report showing title, industry, language, actors, budget, revenue, and profit.

```
WITH finances_normalized AS
(SELECT movie_id, currency, unit,
CASE WHEN LOWER(unit) LIKE
'%billion%' THEN budget * 1000 ELSE
budget END AS budget_m, CASE
WHEN LOWER(unit) LIKE '%billion%'
THEN revenue * 1000 ELSE revenue
END AS revenue_m FROM financials),
actors_agg AS (SELECT ma.movie_id,
GROUP_CONCAT(DISTINCT a.name
ORDER BY a.name SEPARATOR ', ') AS
actors FROM movie_actor ma JOIN
actors a ON ma.actor_id = a.actor_id
GROUP BY ma.movie_id) SELECT
m.movie_id, m.title, m.industry,
l.name AS language,
COALESCE(aa.actors, '') AS actors,
fn.currency, fn.budget_m AS
budget_in_millions, fn.revenue_m AS
revenue_in_millions, (fn.revenue_m -
fn.budget_m) AS profit_in_millions
FROM movies m LEFT JOIN languages
l ON m.language_id = l.language_id
LEFT JOIN actors_agg aa ON
m.movie_id = aa.movie_id LEFT JOIN
finances_normalized fn ON
m.movie_id = fn.movie_id ORDER BY
m.title;
```

Made with GAMMA

Advanced Queries: Part 2

These queries showcase even more advanced SQL concepts including analytical calculations, ranking functions, and complex data transformations that require multiple processing stages and sophisticated logic.

Q6: Highest Revenue Actor

Which actor generates the highest total box-office revenue across all movies they acted in?

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials) SELECT a.actor_id, a.name, ROUND(SUM(fn.revenue_m), 2) AS total_revenue_millions FROM actors a JOIN movie_actor ma ON a.actor_id = ma.actor_id JOIN finances_normalized fn ON ma.movie_id = fn.movie_id GROUP BY a.actor_id, a.name ORDER BY total_revenue_millions DESC LIMIT 1;
```

Q2: Top Actors by Average Rating

Find the top 3 actors with the highest average movie IMDb rating (only actors with ⩾ 2 movies).

```
SELECT a.actor_id, a.name, COUNT(ma.movie_id) AS movies_count, ROUND(AVG(m.imdb_rating), 2) AS avg_imdb_rating FROM actors a JOIN movie_actor ma ON a.actor_id = ma.actor_id JOIN movies m ON ma.movie_id = m.movie_id GROUP BY a.actor_id, a.name HAVING COUNT(ma.movie_id) >= 2 ORDER BY avg_imdb_rating DESC LIMIT 3;
```

Q3: Profitability Buckets

Classify movies into profitability buckets and count movies per bucket.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN budget * 1000 ELSE budget END AS budget_m, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials) SELECT CASE WHEN (fn.revenue_m - fn.budget_m) < 0 THEN 'Loss (<0)' WHEN (fn.revenue_m - fn.budget_m) BETWEEN 0 AND 200 THEN 'Low Profit (0-200M)' WHEN (fn.revenue_m - fn.budget_m) > 200 AND (fn.revenue_m - fn.budget_m) <= 500 THEN 'Medium Profit (200-500M)' WHEN (fn.revenue_m - fn.budget_m) > 500 THEN 'High Profit (>500M)' ELSE 'Unknown' END AS profit_bucket, COUNT(*) AS movies_count FROM finances_normalized fn JOIN movies m ON fn.movie_id = m.movie_id GROUP BY profit_bucket ORDER BY FIELD(profit_bucket,'High Profit (>500M)','Medium Profit (200-500M)','Low Profit (0-200M)','Loss (<0)','Unknown');
```

Q4: Language ROI Analysis

Determine which language produces the highest average ROI.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN budget * 1000 ELSE budget END AS budget_m, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials) SELECT l.language_id, l.name AS language, ROUND(AVG((fn.revenue_m - fn.budget_m) / NULLIF(fn.budget_m,0)), 4) AS avg_roi FROM movies m JOIN finances_normalized fn ON m.movie_id = fn.movie_id LEFT JOIN languages l ON m.language_id = l.language_id WHERE fn.budget_m IS NOT NULL AND fn.budget_m > 0 GROUP BY l.language_id, l.name ORDER BY avg_roi DESC LIMIT 1;
```

Q5: Industry Profit Margin

For each industry, find the movie with the highest profit margin.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN budget * 1000 ELSE budget END AS budget_m, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials), profit_calc AS (SELECT m.movie_id, m.title, m.industry, fn.budget_m, fn.revenue_m, CASE WHEN fn.revenue_m IS NOT NULL AND fn.revenue_m != 0 THEN (fn.revenue_m - fn.budget_m) / fn.revenue_m ELSE NULL END AS profit_margin FROM movies m JOIN finances_normalized fn ON m.movie_id = fn.movie_id) SELECT movie_id, title, industry, ROUND(profit_margin, 4) AS profit_margin FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY industry ORDER BY profit_margin DESC) AS rn FROM profit_calc) t WHERE rn = 1;
```

Made with GAMMA

Advanced Queries: Part 3

These final advanced queries demonstrate the most sophisticated SQL techniques including window functions, recursive logic, network analysis, and comprehensive reporting operations.

Q6: Star Power Score

Compute a "Star Power Score" for each actor = (avg IMDb rating × number of movies). Return top 10.

```
WITH actor_stats AS (SELECT a.actor_id, a.name, COUNT(ma.movie_id) AS movies_count, AVG(m.imdb_rating) AS avg_imdb_rating FROM actors a JOIN movie_actor ma ON a.actor_id = ma.actor_id JOIN movies m ON ma.movie_id = m.movie_id GROUP BY a.actor_id, a.name) SELECT actor_id, name, movies_count, ROUND(avg_imdb_rating, 2) AS avg_imdb_rating, ROUND(avg_imdb_rating * movies_count, 2) AS star_power_score FROM actor_stats ORDER BY star_power_score DESC LIMIT 10;
```

Q7: Actor Collaboration Network

Actor–actor collaboration network: list pairs who co-starred and their collaboration counts.

```
SELECT a1.actor_id AS actor1_id, a1.name AS actor1_name, a2.actor_id AS actor2_id, a2.name AS actor2_name, COUNT(DISTINCT ma1.movie_id) AS co_star_count FROM movie_actor ma1 JOIN movie_actor ma2 ON ma1.movie_id = ma2.movie_id AND ma1.actor_id < ma2.actor_id JOIN actors a1 ON ma1.actor_id = a1.actor_id JOIN actors a2 ON ma2.actor_id = a2.actor_id GROUP BY a1.actor_id, a1.name, a2.actor_id, a2.name ORDER BY co_star_count DESC;
```

Q8: Movie Franchises

Find movie franchises by title prefix and compute total revenue per franchise.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials) SELECT franchise_key, COUNT(*) AS movies_in_franchise, ROUND(SUM(fn.revenue_m), 2) AS total_revenue_millions FROM (SELECT m.movie_id, m.title, CASE WHEN LOCATE(':', m.title) > 0 THEN TRIM(LEFT(m.title, LOCATE(':', m.title)-1)) ELSE SUBSTRING_INDEX(m.title, ' ', 1) END AS franchise_key FROM movies m) t JOIN finances_normalized fn ON t.movie_id = fn.movie_id GROUP BY franchise_key ORDER BY total_revenue_millions DESC LIMIT 20;
```

Q9: Yearly Top Revenue Movies

Rank movies by revenue within each release year — return top-ranked (rank = 1) per year.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m FROM financials) SELECT movie_id, title, release_year, revenue_m, rank_in_year FROM (SELECT m.movie_id, m.title, m.release_year, fn.revenue_m, ROW_NUMBER() OVER (PARTITION BY m.release_year ORDER BY fn.revenue_m DESC) AS rank_in_year FROM movies m JOIN finances_normalized fn ON m.movie_id = fn.movie_id) t WHERE rank_in_year = 1 ORDER BY release_year DESC;
```

Q10: Revenue Deviation Analysis

For each movie, compute absolute and percentage deviation of its revenue from the industry average.

```
WITH finances_normalized AS (SELECT movie_id, CASE WHEN LOWER(unit) LIKE '%billion%' THEN revenue * 1000 ELSE revenue END AS revenue_m, CASE WHEN LOWER(unit) LIKE '%billion%' THEN budget * 1000 ELSE budget END AS budget_m FROM financials), industry_avg AS (SELECT m.industry, AVG(fn.revenue_m) AS industry_avg_revenue FROM movies m JOIN finances_normalized fn ON m.movie_id = fn.movie_id GROUP BY m.industry) SELECT m.movie_id, m.title, m.industry, ROUND(fn.revenue_m,2) AS revenue_m, ROUND(ia.industry_avg_revenue,2) AS industry_avg_revenue, ROUND(fn.revenue_m - ia.industry_avg_revenue,2) AS abs_deviation_m, CASE WHEN ia.industry_avg_revenue = 0 THEN NULL ELSE ROUND((fn.revenue_m - ia.industry_avg_revenue) / ia.industry_avg_revenue * 100, 2) END AS pct_deviation FROM movies m JOIN finances_normalized fn ON m.movie_id = fn.movie_id LEFT JOIN industry_avg ia ON m.industry = ia.industry ORDER BY pct_deviation DESC;
```

Made with GAMMA

Key Takeaways

Build Strong Foundations

Master basic SELECT statements, filtering (WHERE), sorting (ORDER BY), and simple aggregations before moving to advanced concepts.

Understand JOINs

Practice different types of JOINs (INNER, LEFT, RIGHT) to combine data from multiple tables efficiently.

Master Calculations

Learn to perform calculations on data, handle different units (millions vs billions), and use CASE statements for conditional logic.

Learn Window Functions

Understand ROW_NUMBER(), RANK(), and PARTITION BY for advanced analytical queries and ranking operations.

Use CTEs Effectively

Common Table Expressions (WITH clauses) help break complex queries into manageable, readable steps.

Practice Regularly

Work through progressively harder problems and review execution plans to optimize query performance.

Made with GAMMA

Query Performance Tips

Indexing Strategies

- Create indexes on columns used in WHERE, JOIN, and ORDER BY clauses
- Consider composite indexes for multi-column queries
- Avoid over-indexing as it slows down INSERT/UPDATE operations
- Regularly analyze and update statistics

Query Optimization

- Use LIMIT to reduce result set size when possible
- Avoid SELECT * — specify only needed columns
- Use appropriate JOIN types (INNER vs LEFT)
- Consider materialized views for complex aggregations

Made with GAMMA

Common Pitfalls to Avoid

NULL Handling

Always consider NULL values in calculations and comparisons. Use IS NULL/IS NOT NULL and COALESCE() or NULLIF() functions appropriately.

Data Type Mismatches

Ensure consistent data types across JOIN conditions and calculations. Convert units appropriately (millions vs billions).

Cartesian Products

Always specify JOIN conditions to avoid unintentional cross joins that return excessive rows.

Performance Issues

Avoid correlated subqueries in SELECT clause when possible. Use CTEs or temporary tables for complex multi-step operations.

Made with GAMMA

Next Steps

01

Practice with Real Data

Apply these queries to actual movie databases or create your own sample datasets to test different scenarios.

02

Explore Advanced Features

Study window functions, recursive CTEs, and full-text search capabilities in your database system.

03

Learn Query Planning

Understand how to read execution plans and identify performance bottlenecks in your queries.

04

Master Your RDBMS

Dive deeper into vendor-specific features of PostgreSQL, MySQL, SQL Server, or Oracle.

Made with GAMMA